Práctica de Laboratorio 3: Entrada/Salida mediante Sondeo (MMIO)

Richard Vasquez

13 de octubre de 2026

1. Respuestas al Cuestionario

1.1. 1. Organización de la memoria y uso de lw/sw

En el mapeo de E/S en memoria (Memory-Mapped I/O), se reserva un segmento específico del espacio de direcciones físicas de la memoria principal exclusivamente para los registros de control y datos de los periféricos. Generalmente, esta región se ubica en el espacio del sistema (por ejemplo, en MIPS se usa el segmento `kseg1`, típicamente alrededor de la dirección `0xFFFF0000`). La implicación directa es que las instrucciones estándar de lectura y escritura (`lw` y `sw`) se utilizan para comunicarse con el hardware de E/S, en lugar de acceder a la memoria RAM.

1.2. 2. Memory-Mapped I/O vs. E/S por Puertos

La E/S por puertos utiliza un espacio de direcciones aislado y requiere instrucciones dedicadas (como `IN` y `OUT` en x86).

- **Ventajas MMIO:** Reutiliza las mismas instrucciones de memoria, simplificando la Unidad de Control del CPU.
- **Desventajas MMIO:** Consume espacio de direcciones que podría usarse para RAM.
- **Por qué MIPS lo usa:** MIPS es una arquitectura RISC (Conjunto de Instrucciones Reducido). Al usar MMIO, MIPS evita agregar nuevas instrucciones al hardware, manteniendo la pureza y velocidad de decodificación de su filosofía RISC.

1.3. 3. Solapamiento de direcciones

Si dos dispositivos comparten direcciones, ocurrirá una colisión en el bus de datos (corrupción de datos) o comportamiento indefinido de hardware. Esto se evita estrictamente mediante el diseño del *Mapa de Memoria* (Address Decoder), un circuito físico que garantiza que señales de selección de chip (Chip Select) sean mutuamente excluyentes para cada periférico.

1.4. 4. Simplificación del diseño de instrucciones (ISA)

El MMIO permite que el procesador hable con el teclado o el disco duro exactamente como habla con la RAM. Si usara E/S por puertos, MIPS requeriría un nuevo formato de instrucción, nuevos canales en el camino de datos y las instrucciones hipotéticas `in $rt, puerto` y `out puerto, $rt`.

1.5. 5. Funcionamiento del Bus de Datos y Direcciones

Cuando el procesador emite una dirección (ej. `0xFFFF0000`), esta viaja por el Bus de Direcciones. Un componente de hardware llamado decodificador de direcciones (*Address Decoder*) detecta que este rango pertenece a un periférico. Automáticamente, el decodificador envía una señal lógica (Chip Select) al controlador del periférico y deshabilita temporalmente la conexión física con la memoria RAM, asegurando que solo el dispositivo responda en el Bus de Datos.

1.6. 6. Acceso sin privilegios y protección

Normalmente, un programa de usuario no puede acceder a MMIO porque estas direcciones están en un segmento del kernel (`kseg`). La MMU (Memory Management Unit) generará una excepción (Fallo de Segmentación) si lo intenta. Los programas se comunican con el hardware usando `syscalls` (Llamadas al Sistema), dejando que el Sistema Operativo, que tiene permisos, haga el acceso físico.

1.7. 7. Alternativas a la espera activa

Para evitar el uso del "Sondeo"(bucle infinito gastando ciclos de CPU), se emplean: 1. **Interrupciones (Interrupt-Driven I/O):** El hardware avisa al CPU asíncronamente cuando está listo. 2. **DMA (Acceso Directo a Memoria):** Un controlador especial transfiere datos del dispositivo a la RAM sin intervención de la CPU.

1.8. 8. Análisis y Discusión de los Resultados

Se evidenció que la técnica de *Polling* programada en MIPS32 (`beq $t1, $zero, espera`) es funcional pero altamente ineficiente, ya que el procesador queda "secuestradoïterando sobre la misma línea de código, impidiendo el procesamiento de otras tareas (desperdicio de ciclos de reloj). Sin embargo, es fácil de implementar y útil en sistemas dedicados muy simples.